

Flood Monitoring Node

Circuit Diagrams & Pin Connections

Sender (river-gauge field node) and Receiver (LoRa gateway station).
ESP32-WROOM-32 · JSN-SR04T ultrasonic · tipping-bucket rain gauge · SX1276 LoRa 865 MHz.

This document contains, for both boards: a pictorial wiring view drawn as the physical modules, a conventional schematic circuit diagram with standard symbols, and a complete pin-connection table. Build from the tables; use the drawings to find things.

2	System overview — what talks to what
3	Sender — pictorial wiring diagram
4	Sender — schematic circuit diagram
5	Sender — pin connection table
6	Receiver — pictorial wiring diagram
7	Receiver — schematic circuit diagram
8	Receiver — pin connection table
9	Radio settings, build order and common faults
10	Calibration — datum, rain gauge, radio link
11–13	Sender firmware — complete listing
14–15	Receiver firmware — complete listing
16	Libraries, board settings and optional improvements
17	Installing the node — where it goes on the pier
18	Siting the rain gauge
19	Installing the gateway mast, and the survey checklist

ABOUT THE ILLUSTRATIONS

The "real object" views are hand-drawn vector illustrations of the actual modules — the black ESP32 DevKit with its metal-can WROOM module, the blue RA-02 with its gold SMA connector, the green charge board, the sealed ultrasonic transducer. They are drawn rather than photographed so that every pin can be labelled and so the document carries no third-party image rights.

SCHEMATIC CONVENTIONS USED ON PAGES 4 AND 7

Wires that cross without a filled dot are **NOT** connected. A flag marked +3V3, VBAT or a named tag is a net label: every flag with the same name is the same electrical node, even though no wire is drawn between them. Pin positions on the schematic are logical and do not match the physical header order — use the pin tables for that.

1 • System overview

One gateway serves many nodes. Nothing in the field needs a SIM card or mains power.

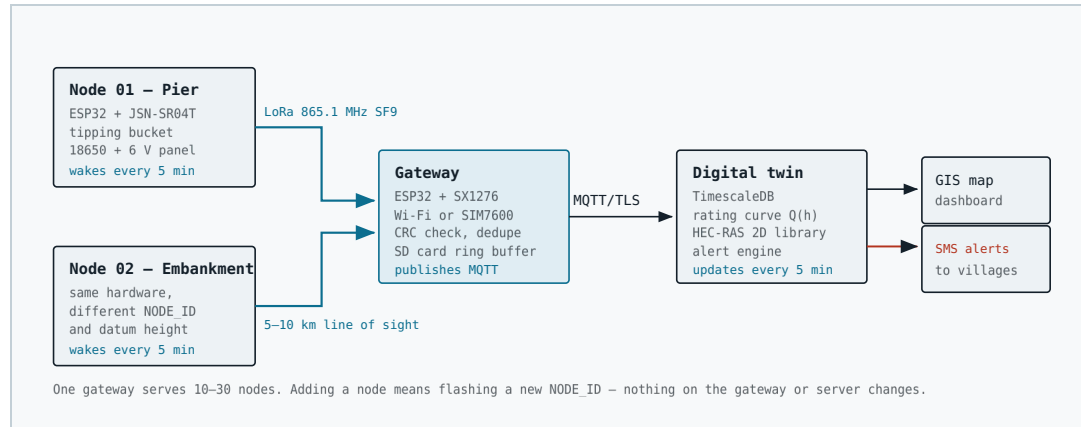


Fig. 1 Field nodes transmit a 13-byte packet every five minutes over LoRa. The gateway validates and forwards them to the server, which drives the dashboard and the SMS alerts.

What each board does

BOARD	POWERED BY	SLEEPS?	JOB
Sender (node)	18650 + 6 V solar	Yes – 10 μ A	Measure water level and rainfall; transmit
Receiver (gateway)	5 V mains / 20 W solar	Never	Listen continuously; buffer to SD; publish over MQTT

Packet format — 13 bytes

BYTE	FIELD	TYPE	MEANING
0	ver	uint8	Protocol version, currently 1
1	node	uint8	Node ID, 1–254
2–3	seq	uint16	Sequence counter — used to drop duplicates
4–5	level_mm	int16	Stage above datum in mm; –1 means no valid echo
6–7	rain_tips	uint16	Bucket tips since the last packet $\times$ 0.2 mm
8–9	batt_mv	uint16	Battery millivolts
10	temp_c	int8	Air temperature, for sound-speed correction
11–12	crc16	uint16	CRC16-CCITT over bytes 0–10

2 · Sender — pictorial wiring

The field node as it physically looks on the bench. Gold pads are pins in use; grey pads stay empty.

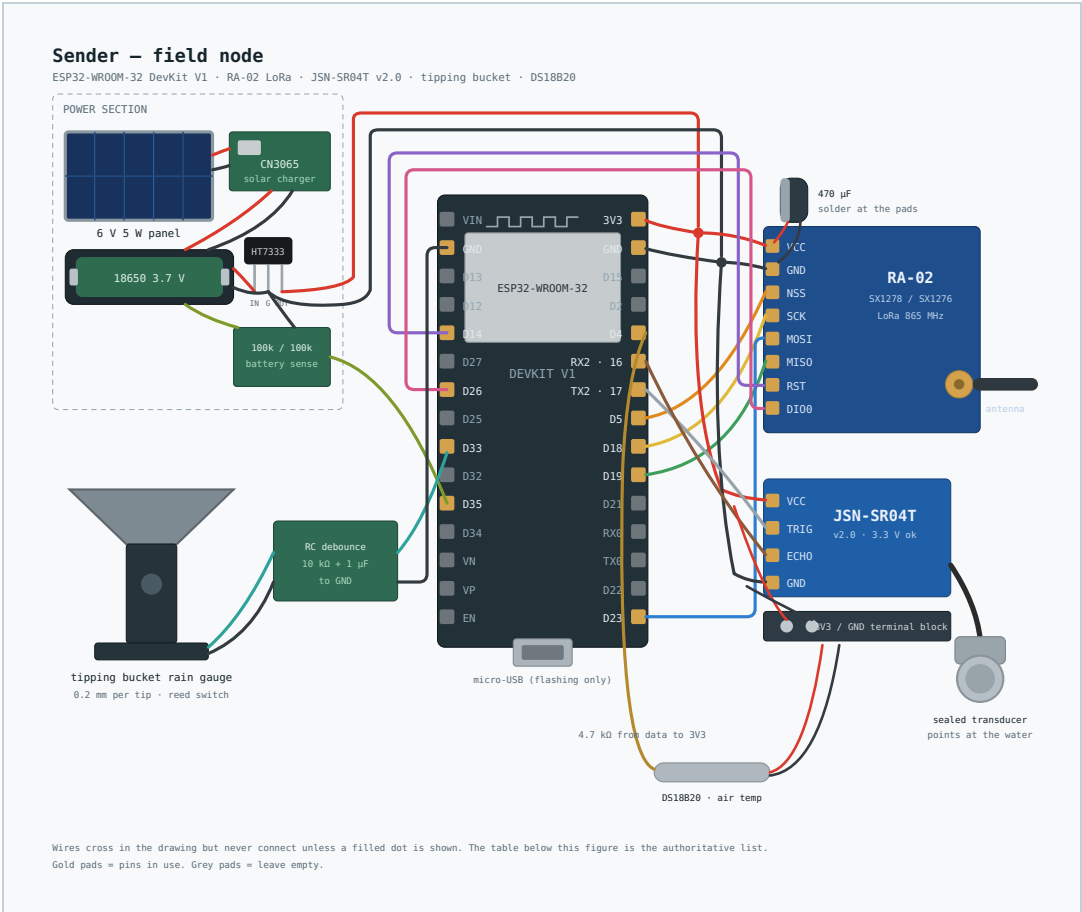


Fig. 2 Sender wiring view. Wires cross in this drawing without connecting; a connection exists only at a pad or a filled dot.

3.3 V GND NSS SCK MISO MOSI RESET DI00 TRIG
ECHO rain batt sense 1-wire

What is in the enclosure

REF	PART	WHERE IT SITS
U1	ESP32-WROOM-32 DevKit V1	On nylon standoffs, clear of the wall
U2	RA-02 LoRa, 865 MHz	Antenna through a bulkhead SMA, never a drilled hole
U3	JSN-SR04T v2.0 driver board	Inside; transducer on 2.5 m cable to the arm
U4	DS18B20 probe	Outside, shaded, out of direct sun
U5 / U6	CN3065 charger, HT7333 regulator	Power section, on the same perfboard
BT1	18650 cell in a holder	Retained so it cannot move in wind
S1	Tipping bucket rain gauge	Outside on a levelled post, shielded cable back to the box

ENCLOSURE DISCIPLINE

Cable glands point downward and every cable gets a drip loop below its gland — water travels along cable jackets and will find any gland that faces up. Two 50 g silica sachets inside, replaced at each service visit.

3 · Sender — schematic circuit diagram

Conventional symbols. Every +3V3 flag is the same node, fed by the power supply panel at the foot of the sheet.

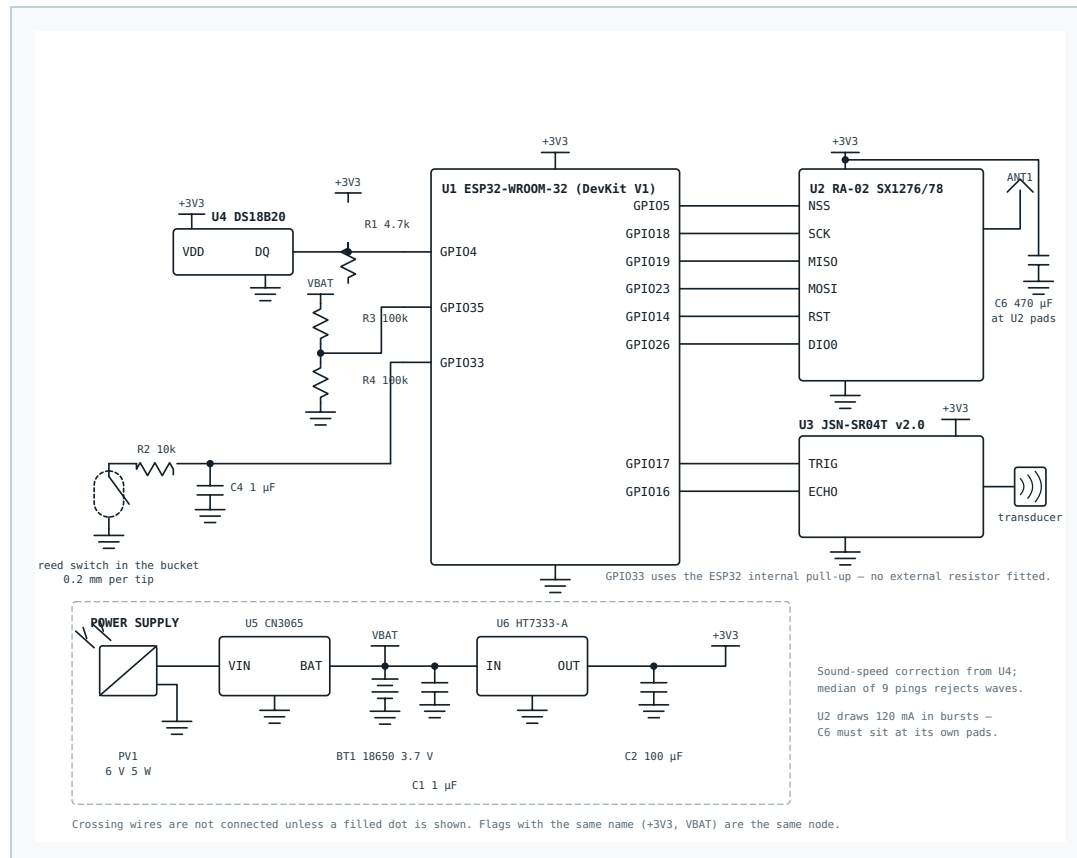


Fig. 3 Sender schematic. The power supply is drawn once at the foot of the sheet; every module then takes its 3.3 V from a net-label flag rather than a drawn rail.

Power budget behind this circuit

STATE	CURRENT	PER 5-MIN CYCLE	CHARGE
Deep sleep	~10 µA	299 s	0.83 µAh
Boot and sensor warm-up	80 mA	0.6 s	13.3 µAh
Nine ultrasonic pings	45 mA	0.5 s	6.3 µAh
LoRa transmit, SF9, 14 dBm	120 mA	0.12 s	4.0 µAh
Total	—	300 s	≈ 24.4 µAh
Per day, 288 cycles	—	—	≈ 7.0 mAh

A 3000 mAh cell holds six to eight weeks of readings with no sun at all once self-discharge and regulator quiescent current are allowed for. The panel covers the months, not the minutes — a week of thick monsoon cloud is a non-event.

THE TWO COMPONENTS PEOPLE LEAVE OUT

C6 must sit at the LoRa module's own pads, not back at the regulator: the 120 mA transmit burst dips the rail otherwise and the node reboots on every packet, a fault that looks exactly like a firmware bug. C4 is equally load-bearing — without it a single bucket tip registers as five or six, and the gauge over-reports rainfall by a factor of several.

4 · Sender — pin connection table

This table is authoritative. Where a drawing disagrees with it, the table is right.

ESP32 connections

ESP32 PIN	SILKSCREEN	GOES TO	WIRE	NOTE
3V3	3V3	HT7333 OUT · LoRa VCC · JSN VCC · DS18B20 VDD	 red	Never feed VIN or USB
GND	GND	Common star ground	 black	One tie point
GPI05	D5	LoRa NSS	 orange	Chip select
GPI018	D18	LoRa SCK	 yellow	SPI clock
GPI019	D19	LoRa MISO	 green	—
GPI023	D23	LoRa MOSI	 blue	Bottom of right header
GPI014	D14	LoRa RST	 purple	—
GPI026	D26	LoRa DIO0	 pink	TX-done interrupt
GPI017	TX2	JSN-SR04T TRIG	 white	10 µs pulse
GPI016	RX2	JSN-SR04T ECHO	 brown	Direct on v2.0; 10k/15k divider on v1.0
GPI033	D33	Rain gauge via R2/C4	 teal	RTC pin — wakes from deep sleep
GPI035	D35	R3/R4 divider midpoint	 olive	Input-only, ADC1, no pull-up
GPI04	D4	DS18B20 DQ	 amber	R1 4.7k pull-up to 3V3

Module pinouts

RA-02 LORA	T0	JSN-SR04T	T0
VCC	3V3 (+ C6 470 µF to GND)	VCC	3V3 (v2.0) or 5 V (v1.0)
GND	GND	TRIG	GPI017
NSS	GPI05	ECHO	GPI016
SCK	GPI018	GND	GND
MOSI	GPI023		
MISO	GPI019	DS18B20	T0
RST	GPI014	VDD (red)	3V3
DIO0	GPI026	DQ (yellow)	GPI04 + 4.7k to 3V3
ANT	3 dBi whip via SMA	GND (black)	GND

Discrete components

REF	VALUE	BETWEEN	PURPOSE
R1	4.7 kΩ	GPI04 — 3V3	1-Wire bus pull-up
R2	10 kΩ	reed switch — GPI033	Series half of the debounce filter
C4	1 µF	GPI033 — GND	Swallows contact bounce (τ = 10 ms)
R3, R4	100 kΩ each	VBAT — GPI035 — GND	Halves 4.2 V into the ADC range
C1	1 µF	HT7333 IN — GND	Regulator input stability
C2	100 µF	HT7333 OUT — GND	Bulk on the 3.3 V rail
C6	470 µF	LoRa VCC — GND	Supplies the 120 mA transmit burst
S1	reed	GPI033 node — GND	Closes once per 0.2 mm of rain

PINS THAT MUST STAY FREE

GPI06–11 are wired to the flash chip — using them stops the board booting. GPIO0, 2, 12 and 15 are strapping pins read at power-up; anything holding GPIO12 high will prevent boot. GPIO34–39 are input-only with no internal pull-up.

5 · Receiver — pictorial wiring

The gateway. Same ESP32, same radio, plus an SD card on the shared SPI bus. No sensors, no sleep.

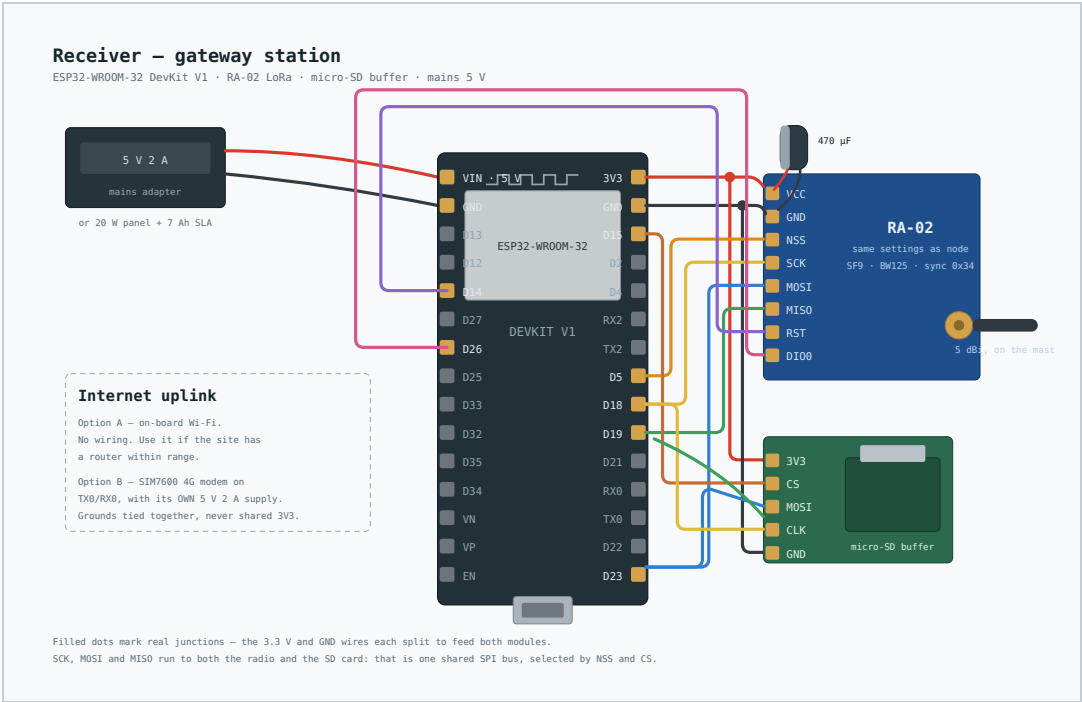


Fig. 4 Receiver wiring view. Filled dots mark real junctions: the 3.3 V and ground wires each split to feed both modules.

WHY 5 V GOES INTO VIN HERE BUT NEVER ON THE SENDER

The DevKit's onboard AMS1117 regulator needs roughly 4.8 V at its input to produce 3.3 V. A mains adapter clears that easily, so on the gateway VIN is the correct entry point. A 3.7 V lithium cell does not, which is why the sender bypasses that regulator and feeds the 3V3 pin directly from an HT7333.

Internet uplink options

OPTION	WIRING	POWER	WHEN TO CHOOSE IT
On-board Wi-Fi	None	Included	Site has a router within range — simplest and cheapest
SIM7600 4G	TX0 / RX0, grounds tied	Its own 5 V 2 A supply	Remote mast with no fixed line; never share the ESP32's 3V3
Ethernet (W5500)	Second SPI bus	From the 3V3 rail	Inside a department building with a wired network

Two additions before this goes on a tower

- Write every payload to the SD card as well as publishing it, so a dropped link loses nothing.
- Publish a heartbeat on flood/GW-ID/status every minute, so the server can tell a silent gateway from a dry season.

6 • Receiver — schematic circuit diagram

SCK, MOSI and MISO are one shared bus. The two devices are told apart by their separate chip-select lines.

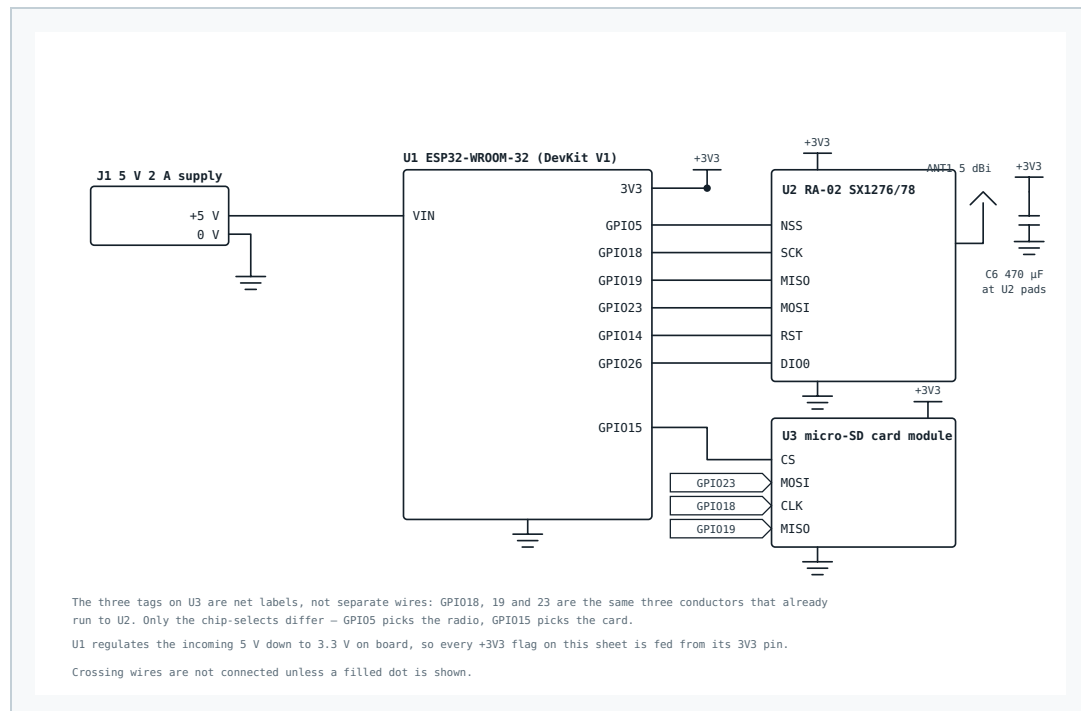


Fig. 5 Receiver schematic. Drawing the shared bus as net labels rather than tapped wires keeps the three SPI conductors readable — they are still physically three wires reaching both modules.

How one bus serves two devices

SPI is a bus, not a point-to-point link. Clock, MOSI and MISO are wired in parallel to every device; each device additionally gets its own chip-select line, and a device only listens and only drives MISO while its own chip-select is held low. The firmware therefore pulls GPIO5 low to talk to the radio and GPIO15 low to talk to the card, never both at once.

WHAT THE NET LABELS MEAN ON THIS SHEET

The three tags on U3 carry the names GPIO18, GPIO19 and GPIO23 — the same nets already drawn from U1 to U2. Drawing them as tags rather than as tapped wires avoids five meaningless crossings; physically they are still three conductors, each reaching both modules.

THE FAILURE THIS CAUSES IN PRACTICE

If LoRa packets turn to garbage the moment the SD card is fitted, the card's module is holding MISO driven while its chip-select is high. Fit a passive SD breakout, or move the card to the HSPI bus: GPIO12 MISO, GPIO13 MOSI, GPIO25 CLK, chip-select on GPIO27.

7 · Receiver — pin connection table

ESP32 PIN	SILKSCREEN	GOES TO	WIRE	NOTE
VIN	VIN	5 V supply positive	 red	Correct here — onboard regulator has headroom
GND	GND	5 V supply negative + common ground	 black	—
3V3	3V3	LoRa VCC + SD 3V3	 red	Junction; output of the onboard regulator
GPI05	D5	LoRa NSS	 orange	Radio chip select
GPI015	D15	SD CS	 rust	Card chip select — must differ from NSS
GPI018	D18	LoRa SCK + SD CLK	 yellow	Shared bus
GPI023	D23	LoRa MOSI + SD MOSI	 blue	Shared bus
GPI019	D19	LoRa MISO + SD MISO	 green	Shared bus
GPI014	D14	LoRa RST	 purple	—
GPI026	D26	LoRa DIO0	 pink	RX-done interrupt
TX0 / RX0	TX0 / RX0	SIM7600 modem, if used	—	Modem needs its own 5 V 2 A supply; tie grounds

Module pinouts

RA-02 LORA	T0	MICRO-SD MODULE	T0
VCC	3V3 + C6 470 μ F	3V3	3V3
GND	GND	CS	GPI015
NSS	GPI05	MOSI	GPI023
SCK	GPI018	CLK	GPI018
MOSI	GPI023	MISO	GPI019
MISO	GPI019	GND	GND
RST	GPI014		
DIO0	GPI026		
ANT	5 dBi collinear on the mast		

IF PACKETS TURN TO GARBAGE THE MOMENT YOU FIT THE SD CARD

Some inexpensive SD modules carry a level-shifter that keeps driving MISO even when their chip-select is high, which corrupts every read the radio makes. Either use a plain passive SD breakout, or move the card to the second SPI bus — HSPI on GPIO12 (MISO), GPIO13 (MOSI), GPIO25 (CLK) with chip-select on GPIO27, since GPIO14 is already taken by the radio reset.

8 • Radio settings, build order, common faults

Radio settings — identical on both boards

If any one of these differs between sender and receiver, the gateway hears nothing at all. There is no error message and no warning; the link is simply silent.

SETTING	VALUE	WHY
Frequency	865.1 MHz	Inside India's licence-free 865–867 MHz band
Spreading factor	SF9	~90 ms airtime for 13 bytes; 5–10 km in flat terrain
Bandwidth	125 kHz	Standard for this band
Coding rate	4/5	—
Sync word	0x34	Hardware filter — your network ignores other traffic
TX power	14 dBm	Within the module's rating without a heatsink
Preamble	8 symbols	Default

Target link margin: RSSI better than -110 dBm and SNR above -7 dB once installed. Anything weaker works on a clear day and disappears in heavy rain — which is exactly when the system matters.

Build order

1. Power section alone. Measure $3.30\text{ V} \pm 0.05$ at the HT7333 output with the cell at half charge. Do not go further until this reads correctly.
2. Add the ESP32. Flash a blink sketch, then measure deep-sleep current — under $100\text{ }\mu\text{A}$ on a modified DevKit, under $20\text{ }\mu\text{A}$ on a bare module.
3. Add the LoRa module with its antenna fitted and C6 soldered at its pads. Test a link across the bench, then at 50 m.
4. Add the ultrasonic sensor. Point it at the floor and check the reading against a tape measure.
5. Add the rain gauge and the temperature probe. Tip the bucket by hand ten times; the counter must read exactly ten.
6. Run the complete node on battery for 72 hours indoors before it goes anywhere near the river.

Faults and what they look like

SYMPTOM	CAUSE	FIX
Node reboots on every transmission	C6 missing, too small, or mounted at the regulator instead of the module	$470\text{ }\mu\text{F}$ at the LoRa pads
Gateway receives nothing, ever	One radio setting differs between the boards	Compare all seven settings above
Deep sleep draws 8 mA	DevKit USB-serial chip and power LED stay awake	Cut the LED trace, or build on a bare WROOM module
Rain counted on a dry night	Induced pulses on an unshielded gauge cable	Shielded pair, shield grounded at the ESP32 end only
Water level flatlines	Sensor seeing the pier face or a spider web	Realign the arm; fit a stilling tube
Battery falls and never recovers	Panel shaded, dirty or facing wrong	At noon, open-circuit panel voltage should exceed 6 V
Board will not enter flash mode	Something loading a strapping pin at power-up	Disconnect peripherals from GPIO0, 2, 12, 15 while flashing

BEFORE YOU SOLDER ANYTHING PERMANENT

Build both boards on breadboard and get one packet across the room first. A bad jumper wire looks exactly like a firmware bug, and discovering that on a bridge pier costs a day.

9 • Calibration

Three numbers decide whether the data is evidence or noise: the datum height, the millimetres per bucket tip, and the link margin. Measure all three on installation day and write them in the log at the foot of this page.

Ultrasonic — finding DATUM_MM

1. Mount the arm and sensor. Check with a spirit level that the transducer face is horizontal. A 5° tilt adds 15 mm of error at 4 m, and the error grows as the water drops.
2. Read the site staff gauge. Example: it reads **2.140 m**.
3. Put the node in test mode — comment out the `goToSleep()` call and print `dist` over serial. Example: it reports **4110 mm**.
4. $\text{DATUM_MM} = \text{staff} + \text{dist} = 2140 + 4110 = \mathbf{6250}$. Write that number into the firmware, on the inside of the enclosure lid, and in the site log.
5. Repeat a week later at a different water level. If `staff + dist` gives the same constant within ± 20 mm, the installation is sound. If it drifts, the arm is sagging.

Rain gauge — verifying MM_PER_TIP

The datasheet says 0.2 mm. Believe it only after you measure it. For a funnel of diameter D mm, one tip should take $V = (\pi/4) \times D^2 \times 0.2 / 1000$ millilitres. A 200 mm funnel gives **6.28 ml per tip**.

1. Level the gauge using its built-in bubble level. This matters more than anything else in the procedure.
2. Measure exactly 200 ml with a syringe or a lab measuring cylinder.
3. Drip it into the funnel slowly — 20 to 30 minutes, matching a real rain rate. Pouring fast under-counts, because water passes through while the bucket is mid-tip.
4. Count the tips on the serial monitor. 200 ml $\div$ 6.28 ml should give about **32 tips**.
5. $\text{MM_PER_TIP} = 0.2 \times 32 / (\text{tips actually counted})$. Counted 35? The true figure is **0.183 mm**, not 0.2.

RE-CHECK AFTER THE FIRST MONTH

A new bucket's bearings bed in and the count shifts. Repeat the pour test at the first service visit, then quarterly. A stage record without a calibration history is not admissible evidence for a flood study.

Radio — proving the link before you climb anything

Load a continuous-transmit sketch into the node, put it in a bag, and walk away from the gateway while printing RSSI and SNR at the receiving end. The installed link should sit at **RSSI better than -110 dBm and SNR above -7 dB** at SF9. Anything weaker works on a clear day and vanishes in heavy rain — precisely when the system matters. If the margin is short: raise the gateway antenna (height beats power, every time), move to SF10 or SF11, or add a relay node.

Site calibration log

DATE	NODE	STAFF GAUGE (M)	REPORTED DIST (MM)	DATUM_MM	TIPS / 200 ML	MM_PER_TIP	RSSI

10 · Sender firmware

One file. Change NODE_ID, DATUM_MM and MM_PER_TIP per station; everything else is identical across the fleet.

NODE/NODE.INO – PART 1 OF 3: CONFIGURATION, PINS, CRC

```
// =====
// FLOOD NODE – SENDER (transmitter)
// ESP32-WROOM-32 + J5N-SR04T v2.0 + tipping bucket + DS18B20 + SX1276
// Arduino IDE. One file. Every cycle ends in deep sleep.
// =====

#include <SPI.h>
#include <LoRa.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include "driver/rtc_io.h"

// ----- per-node configuration: change these three per station ----
#define NODE_ID      1          // unique per node, 1..254
#define DATUM_MM     6250      // sensor face above datum, measured on site
#define MM_PER_TIP   0.2f      // from the rain gauge calibration (page 10)

#define SLEEP_SECONDS 300      // 5 minutes between transmissions

// ----- pin map (matches the tables on page 5) -----
#define PIN_TRIG     17        // TX2 on the silkscreen
#define PIN_ECHO     16        // RX2 on the silkscreen
#define PIN_RAIN     GPIO_NUM_33 // RTC pin - can wake the chip
#define PIN_TEMP     4
#define PIN_VBAT     35        // input-only, ADC1
#define LORA_SS      5
#define LORA_RST     14
#define LORA_DIO0    26

// ----- radio: must match the gateway exactly -----
#define LORA_FREQ    865100000L // IN865 licence-free band
#define LORA_SF      9
#define LORA_BW      125E3
#define LORA_SYNC    0x34

// ----- these survive deep sleep -----
RTC_DATA_ATTR uint16_t rainTips = 0;
RTC_DATA_ATTR uint16_t seq      = 0;

OneWire oneWire(PIN_TEMP);
DallasTemperature tempSensor(&oneWire);

// ----- CRC16-CCITT. The gateway runs the identical function -----
uint16_t crc16(const uint8_t *d, size_t n) {
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < n; i++) {
        crc ^= (uint16_t)d[i] << 8;
        for (uint8_t b = 0; b < 8; b++)
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
    }
    return crc;
}
```

```
// ----- one ultrasonic ping: returns echo width in microseconds ---
uint32_t pingOnce() {
  digitalWrite(PIN_TRIG, LOW); delayMicroseconds(4);
  digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  return pulseIn(PIN_ECHO, HIGH, 40000UL); // 40 ms ceiling = about 6.8 m
}

int cmpU32(const void *a, const void *b) {
  uint32_t x = *(const uint32_t*)a, y = *(const uint32_t*)b;
  return (x > y) - (x < y);
}

// ----- median of 9 pings, corrected for air temperature -----
// A single ping lands on whatever the surface was doing at that instant.
// The median throws out both the wave crest and the spray dropout;
// a mean would not.
int16_t measureDistanceMM(float tempC) {
  uint32_t s[9];
  uint8_t n = 0;
  for (uint8_t i = 0; i < 9; i++) {
    uint32_t t = pingOnce();
    if (t > 0) s[n++] = t;
    delay(60); // let the echoes die away
  }
  if (n < 5) return -1; // too few valid returns
  qsort(s, n, sizeof(uint32_t), cmpU32);
  uint32_t median = s[n / 2];

  float c = 331.4f + 0.606f * tempC; // speed of sound, m/s
  float mm = (median * c) / 2000.0f; // us x m/s / 2 -> mm
  if (mm < 250.0f || mm > 4500.0f) return -1; // blind zone / out of range
  return (int16_t)(mm + 0.5f);
}

// ----- battery, through the 100k/100k divider on GPIO35 -----
uint16_t readBatteryMV() {
  analogSetPinAttenuation(PIN_VBAT, ADC_11db);
  uint32_t acc = 0;
  for (uint8_t i = 0; i < 16; i++) { acc += analogReadMilliVolts(PIN_VBAT); delay(2); }
  return (uint16_t)((acc / 16) * 2); // the divider halved it
}
```

```
// ----- build and send the 13-byte packet (format on page 2) -----
void sendPacket(int16_t levelMM, int8_t tempC, uint16_t battMV) {
  uint8_t p[13];
  p[0] = 1; // protocol version
  p[1] = NODE_ID;
  p[2] = seq >> 8; p[3] = seq & 0xFF;
  p[4] = levelMM >> 8; p[5] = levelMM & 0xFF;
  p[6] = rainTips >> 8; p[7] = rainTips & 0xFF;
  p[8] = battMV >> 8; p[9] = battMV & 0xFF;
  p[10] = (uint8_t)tempC;
  uint16_t c = crc16(p, 11);
  p[11] = c >> 8; p[12] = c & 0xFF;

  LoRa.beginPacket();
  LoRa.write(p, 13);
  LoRa.endPacket(); // blocks until sent
  seq++;
}

// ----- arm both wake sources and sleep -----
void goToSleep() {
  LoRa.sleep();
  rtc_gpio_pullup_en(PIN_RAIN); // hold the line high in sleep
  esp_sleep_enable_ext0_wakeup(PIN_RAIN, 0); // wake when a tip pulls it low
  esp_sleep_enable_timer_wakeup((uint64_t)SLEEP_SECONDS * 1000000ULL);
  esp_deep_sleep_start();
}

void setup() {
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(33, INPUT_PULLUP); // no external pull-up fitted

  esp_sleep_wakeup_cause_t why = esp_sleep_get_wakeup_cause();

  // --- a rain tip: count it and go straight back to sleep -----
  if (why == ESP_SLEEP_WAKEUP_EXT0) {
    rainTips++;
    delay(150); // let the contact settle
    while (digitalRead(33) == LOW) delay(10); // wait for the bucket to finish
    delay(100);
    goToSleep();
  }

  // --- timer wake, or first boot: measure and transmit -----
  tempSensor.begin();
  tempSensor.requestTemperatures();
  float tC = tempSensor.getTempCByIndex(0);
  if (tC < -40 || tC > 80) tC = 25.0; // probe missing -> assume 25 C

  int16_t dist = measureDistanceMM(tC);
  int16_t level = (dist < 0) ? -1 : (int16_t)(DATUM_MM - dist);

  SPI.begin(18, 19, 23, LORA_SS);
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DI00);
  if (LoRa.begin(LORA_FREQ)) {
    LoRa.setSpreadingFactor(LORA_SF);
    LoRa.setSignalBandwidth(LORA_BW);
    LoRa.setCodingRate4(5);
    LoRa.setSyncWord(LORA_SYNC);
    LoRa.setTxPower(14);
    sendPacket(level, (int8_t)tC, readBatteryMV());
  }

  rainTips = 0; // counter reported, start the next interval
  goToSleep();
}

void loop() { } // never reached - every cycle ends in sleep
```

WHY RAINTIPS RESETS AFTER SENDING RATHER THAN ON A SCHEDULE

The node reports tips *since the last packet*, and the server divides by the elapsed time to get a rate. A lost packet then costs one interval's resolution but never loses the rain itself — the next packet's count still covers the gap. A node sending a running total would silently lose everything on a counter reset.

11 · Receiver firmware

Never sleeps. Parked in continuous receive, validating and forwarding whatever arrives.

GATEWAY/GATEWAY.INO – PART 1 OF 2: CONFIGURATION, CRC, SD LOG

```
// =====
// FLOOD GATEWAY – RECEIVER
// ESP32-WROOM-32 + SX1276 + micro-SD. Never sleeps.
// Listens continuously, validates, de-duplicates, publishes, logs.
// =====

#include <WiFi.h>
#include <PubSubClient.h>
#include <SPI.h>
#include <LoRa.h>
#include <SD.h>

const char* WIFI_SSID = "site-wifi";
const char* WIFI_PASS = "*****";
const char* MQTT_HOST = "twin.example.in";
const int MQTT_PORT = 1883;
const char* GW_ID = "GW-KAMRUP-01";

#define LORA_SS 5
#define LORA_RST 14
#define LORA_DIO0 26
#define SD_CS 15 // shares SCK/MOSI/MISO with the radio

WiFiClient net;
PubSubClient mqtt(net);

uint16_t lastSeq[255]; // per-node duplicate filter
bool sdReady = false;
unsigned long lastBeat = 0;

// ----- identical to the node's function -----
uint16_t crc16(const uint8_t *d, size_t n) {
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < n; i++) {
        crc ^= (uint16_t)d[i] << 8;
        for (uint8_t b = 0; b < 8; b++)
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
    }
    return crc;
}

void connectAll() {
    if (WiFi.status() != WL_CONNECTED) {
        WiFi.begin(WIFI_SSID, WIFI_PASS);
        for (int i = 0; i < 40 && WiFi.status() != WL_CONNECTED; i++) delay(500);
    }
    if (!mqtt.connected()) mqtt.connect(GW_ID);
}

// ----- every payload also goes to the card, link up or down -----
void logToCard(const char *line) {
    if (!sdReady) return;
    File f = SD.open("/log.csv", FILE_APPEND);
    if (!f) return;
    f.println(line);
    f.close();
}
```

```

void setup() {
  Serial.begin(115200);
  memset(lastSeq, 0xFF, sizeof(lastSeq));

  // One SPI bus, two devices. Start the radio first, then the card.
  SPI.begin(18, 19, 23, LORA_SS);
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DI00);
  while (!LoRa.begin(865100000L)) { Serial.println("LoRa init failed"); delay(2000); }
  LoRa.setSpreadingFactor(9);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
  LoRa.setSyncWord(0x34);
  LoRa.receive(); // park in continuous receive

  sdReady = SD.begin(SD_CS, SPI);
  Serial.println(sdReady ? "SD ready" : "SD missing - buffering disabled");

  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  connectAll();
}

void loop() {
  mqtt.loop();
  if (!mqtt.connected()) connectAll();

  // --- heartbeat, so a silent gateway is distinguishable from no rain -
  if (millis() - lastBeat > 60000UL) {
    lastBeat = millis();
    char t[48];
    snprintf(t, sizeof(t), "flood/%s/status", GW_ID);
    mqtt.publish(t, "{\"alive\":true}");
  }

  int sz = LoRa.parsePacket();
  if (sz != 13) return; // wrong length: not one of ours

  uint8_t p[13];
  for (int i = 0; i < 13; i++) p[i] = LoRa.read();

  uint16_t got = ((uint16_t)p[11] << 8) | p[12];
  if (got != crc16(p, 11)) return; // corrupt in the air, drop silently

  uint8_t node = p[1];
  uint16_t seq = ((uint16_t)p[2] << 8) | p[3];
  if (seq == lastSeq[node]) return; // duplicate
  lastSeq[node] = seq;

  int16_t level = (int16_t)(((uint16_t)p[4] << 8) | p[5]);
  uint16_t tips = ((uint16_t)p[6] << 8) | p[7];
  uint16_t batt = ((uint16_t)p[8] << 8) | p[9];
  int8_t tempC = (int8_t)p[10];

  char topic[48], payload[220];
  snprintf(topic, sizeof(topic), "flood/%s/node/%u", GW_ID, node);
  snprintf(payload, sizeof(payload),
    "{\"node\":\"%u\",\"seq\":\"%u\",\"level_mm\":\"%d\",\"rain_tips\":\"%u\",",
    "\"rain_mm\":\"%.1f\",\"batt_mv\":\"%u\",\"temp_c\":\"%d\",\"rssi\":\"%d\",\"snr\":\"%.1f\"}",
    node, seq, level, tips, tips * 0.2f, batt, tempC,
    LoRa.packetRssi(), LoRa.packetSnr());

  mqtt.publish(topic, payload);
  logToCard(payload);
  Serial.println(payload);
}

```

12 · Libraries, board settings, improvements

Arduino IDE setup

ITEM	VALUE	WHERE
Board package	esp32 by Espressif Systems	Boards Manager
Board	ESP32 Dev Module	Tools → Board
Upload speed	921600	Tools
Flash frequency	80 MHz	Tools
Partition scheme	Default 4 MB with spiffs	Tools
Core debug level	None	Tools — keeps the serial port quiet

Libraries to install

LIBRARY	AUTHOR	USED BY
LoRa	Sandeep Mistry	Both boards
OneWire	Paul Stoffregen	Sender — DS18B20
DallasTemperature	Miles Burton	Sender — DS18B20
PubSubClient	Nick O'Leary	Receiver — MQTT
SD, SPI, WiFi	bundled with the ESP32 core	Receiver

Optional improvements worth making early

FRAGMENTS — DROP INTO THE FILES ABOVE

```
// ----- 1. spread the nodes out so they stop colliding -----
// Every node wakes on a 5-minute boundary, so eventually two transmit at
// the same instant and both packets are lost. Give each node a fixed slot
// plus a little jitter. Replace the timer line inside goToSleep():

uint64_t sleepUs = (uint64_t)SLEEP_SECONDS * 1000000ULL
                  + (uint64_t)NODE_ID * 1700000ULL    // fixed slot per node
                  + (uint64_t)(esp_random() % 3000000ULL); // +0..3 s jitter
esp_sleep_enable_timer_wakeup(sleepUs);

// ----- 2. interrupt-driven receive on the gateway -----
// Handles the packet the moment DIO0 fires instead of whenever loop()
// next comes round. Add these, and keep the length and CRC checks.

volatile bool gotPacket = false;
volatile int  pktSize   = 0;

void onReceive(int sz) { pktSize = sz; gotPacket = true; }

// in setup(), after the radio settings:
LoRa.onReceive(onReceive);
LoRa.receive();

// in loop():
if (gotPacket) {
  gotPacket = false;
  if (pktSize == 13) { /* read 13 bytes, check CRC, publish, log */ }
  LoRa.receive();      // re-arm. Forget this and it stops dead.
}
```

BEFORE FLASHING

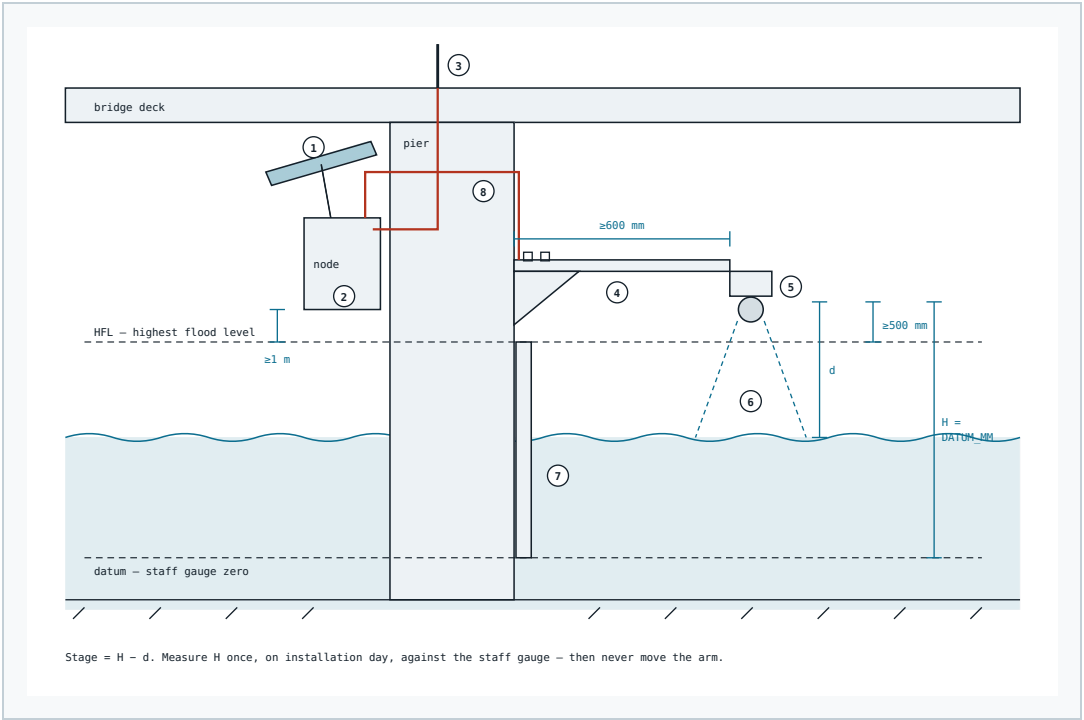
Disconnect anything loading GPIO0, 2, 12 or 15 while you upload. Those are strapping pins read at power-up, and a sensor holding one of them will leave the board refusing to enter flash mode — which looks like a broken USB cable.

TEST THE TWO BOARDS ON A BENCH BEFORE THE MAST

Flash both, put them a metre apart with antennas fitted, and watch a packet arrive on the gateway's serial monitor. Only then take the ultrasonic reading against a tape measure, and only then tip the bucket ten times and check the counter reads exactly ten.

13 · Installing the node

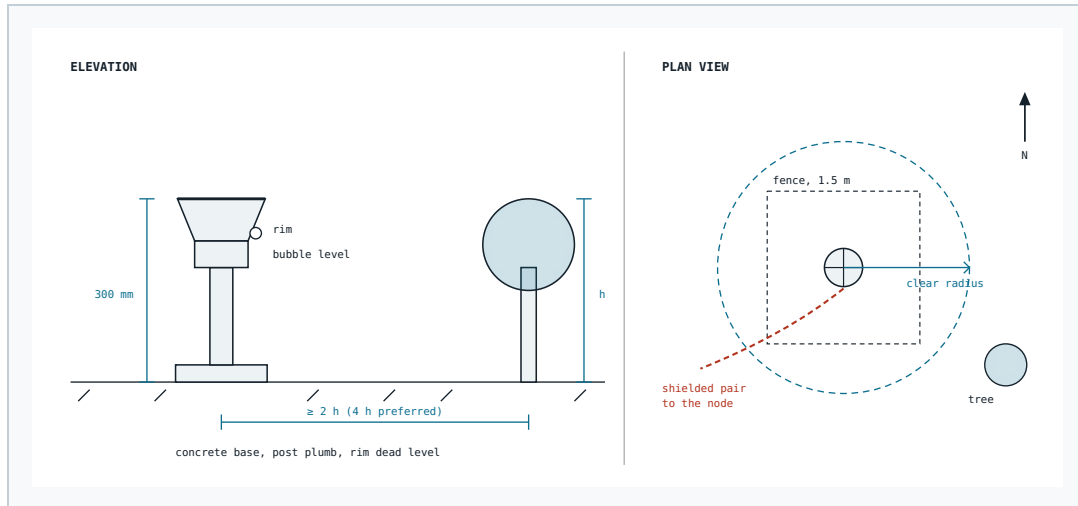
Side elevation of a typical bridge-pier installation. Every dimension below is a minimum, not a target.



#	ITEM	RULE
1	Solar panel	Tilt about 26° facing south, angled so monsoon rain washes it. Clean at every visit — a month of dust halves the output.
2	Enclosure	On the downstream face, at least 1 m above the highest flood mark you can find on the pier. Glands face down.
3	Antenna	Vertical, as high as you can reach — above the parapet if possible. Height beats transmit power every time.
4	Cantilever arm	Galvanised, U-bolted to the pier with a gusset. At least 600 mm clear of the face so echoes come off water, not concrete.
5	Transducer	Face horizontal — check with a spirit level. At least 500 mm above HFL: 250 mm of that is the sensor's blind zone.
6	Measurement cone	Roughly 15° wide. Keep it clear of the pier, the arm and any debris line. A stilling tube helps in choppy water.
7	Staff gauge	Use the existing one if there is one, so your readings can be compared with CWC records. This is what you calibrate against.
8	Cable in conduit	Clipped to the pier face, with a drip loop below every gland. Never leave cable hanging in the flow.

14 · Siting the rain gauge

The gauge is the one instrument whose accuracy is decided almost entirely by where you put it. A perfectly calibrated gauge under a tree measures the tree.



The four rules that matter

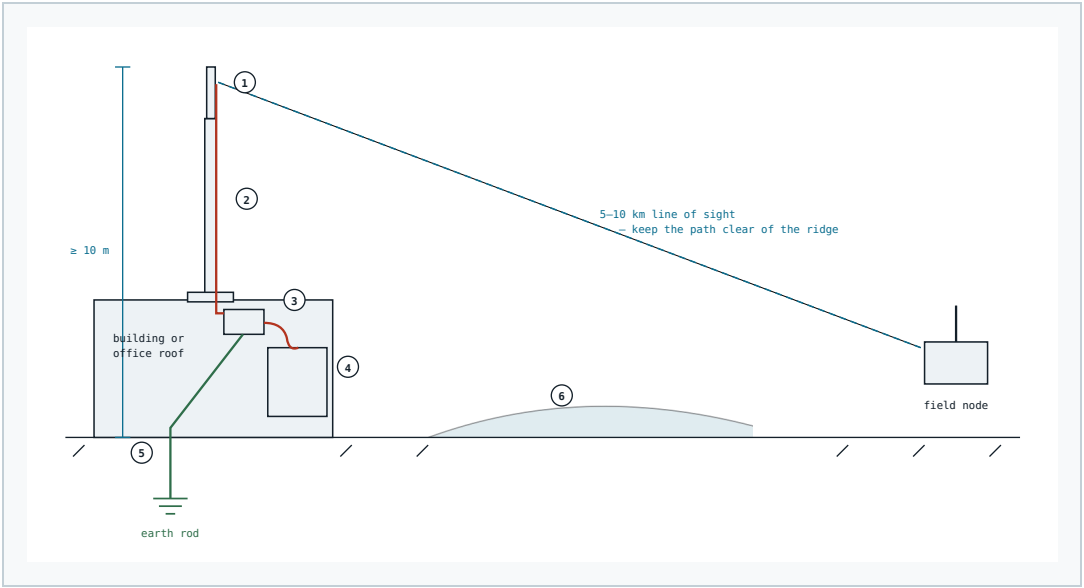
1. **Clear of obstructions.** Any tree, wall or building must stand at least twice its own height away from the gauge — four times is better. If a 6 m tree is 8 m away, move the gauge, not the tree.
2. **Rim at 300 mm above ground,** on a plumb post set in a concrete base. Too low and splash from the ground enters the funnel; too high and wind carries drops past the rim.
3. **Rim dead level.** Use the built-in bubble. A tilted gauge over-collects on one side and the bucket tips at the wrong volume — this ruins the calibration you did on page 10.
4. **Shielded cable back to the node,** shield grounded at the ESP32 end only, routed away from any mains or pump wiring. An unshielded run beside a pump counts phantom tips every time the pump starts.

IF THE ONLY AVAILABLE SPOT IS EXPOSED

On a windy embankment a gauge under-catches, sometimes by 10–20 % in driving rain. A turf wall or a slatted wind shield around the gauge at rim height recovers most of it. Note the shielding in the site log, because it changes how your figures compare with an unshielded IMD gauge nearby.

15 · Installing the gateway

The gateway hears every node, so its antenna height sets the range of the whole network.



#	ITEM	RULE
1	Antenna	5 dBi collinear, vertical, at the very top of the mast. Doubling the height gains far more range than doubling the power.
2	Coax run	Keep it under 5 m in RG58, or use LMR-200 for longer. Every metre of thin coax at 868 MHz costs you range.
3	Lightning arrestor	Inline on the coax at the point it comes off the mast, bonded to the earth conductor. Assam gets a lot of lightning.
4	Enclosure	On the wall, out of the sun, with a drip loop before the cable entry.
5	Earth rod	Its own rod, bonded to the arrestor and to the mast. Do not rely on the building's neutral.
6	Path clearance	Walk or map the line to each node. A ridge or a row of tall trees midway will cut a link that looks fine on paper.

Installation-day checklist

✓	STEP	EVIDENCE TO RECORD
	Gateway mounted, antenna vertical, arrestor earthed	Photo, mast height
	Node enclosure and arm fixed; transducer face levelled	Spirit-level photo
	Staff gauge read and reported distance logged	DATUM_MM written into firmware and on the lid
	Rain gauge levelled, obstruction distances paced out	Sketch of the clear radius
	First packet received at the gateway	RSSI and SNR figures
	Bucket tipped ten times by hand	Counter reads exactly ten
	Panel open-circuit voltage checked at midday	Volts, above 6 V
	Silica gel fitted, glands tightened, box closed	Date on the service label

LEAVE THE SITE WITH A PHOTOGRAPH OF EVERY DIMENSION

Six months later, when a reading drifts, the only way to tell a sagging arm from a real change in the river is a photograph of how it looked on day one. Take pictures of the arm, the staff gauge with the water level visible, and the rain gauge with its surroundings.